

Indoor Navigation – README for Reviewer

(Key Highlights)

Project Summary

Android indoor navigation app combining Wi-Fi RSSI localization, A* pathfinding, and a dynamic building model with points of interest (POIs). The UI provides step-by-step guidance.

Code Structure

- core/ – Core cross-platform logic: graph representation, A* pathfinding, CAD Automation, Positioning, Statistic models, machine learning models.
- ui/ – Client app (Flutter/Dart): screens, map rendering, navigation instructions, app state.
- server/ – Backend services (Python): endpoints for syncing building/POIs, processing RSSI data, and integration layer.
- res/ – Resources (maps, configuration, sample data).
- Other files – run.py to start backend, requirements.txt for Python dependencies, workshop specification doc.

How It Works (High-Level)

- Location estimation via Wi-Fi RSSI measurements.
- Building graph with nodes (halls/doors/POIs) and edges with weight (distance, constraints).
- Pathfinding using A* with support for dynamic updates (e.g., temporary blockages).
- UI presents directions and highlights POIs.

Key Highlights to Review

- A* pathfinding: heuristic choice (admissibility), dynamic weights, performance on larger maps – core/ManagerFloor.py.
- Wi-Fi RSSI localization: smoothing/filtering, AP mapping, cross-device calibration – core/predict.
- The UI integrates basic PDR to visualize short-term motion and keep the user's on-screen position responsive between Wi-Fi fixes – ui/lib/screens/user/navigate_screen.dart.
- HMM-based localization: sequential state estimation with Viterbi decoding for smoothing noisy Wi-Fi RSSI inputs – core/predict/hmm_model.py.
- CAD file automation: parsing and converting building CAD data into navigable graph structures with nodes, edges, and POIs – core/app.py.
- Bitmap: raster image parsing and conversion into bitmap for building navigable graph – core/bitmap.py.

How to Run

- Server (Python): install dependencies from requirements.txt and run `python run.py`.
- Client (UI): open ui/ (Flutter/Dart), run on emulator/device; grant Wi-Fi/location/sensor permissions.